

Formulario Python

Sintaxis básica y aspectos generales

```
x = 5
print(x)

if __name__ == "__main__":
    x = 5
    print(x)
```

Python es un lenguaje interpretado, lo que quiere decir que este compila y ejecuta el código línea por línea.

Al ser un lenguaje fuertemente orientado a objetos, TODO se considera como uno, incluyendo las funciones.

La diferencia entre el script directo y el que está dentro del if es la prioridad de ejecución. El código dentro del if sólo se ejecuta si se corre directamente el archivo, más no lo hará si el archivo se usa para una importación.

Tipos de datos

Tipo	Ejemplo	Descripción	Mutabilidad
int	i = 5	Números enteros	NO
float	i = 3.14	Números de punto flotante	NO
complex	x = 2 + 3j	Números complejos	NO
bool	x = True	Valores booleanos (True/False)	NO
str	s = "Holaaa"	Cadenas de texto	NO
tuple	t = (1,2,3)	Tuplas	NO
frozenset	frozenset()	falta de descripción	NO
bytes	b = b"abc"	Obtiene la representación en bytes de lo que esté entre comillas	NO
NoneType	None	None	NO
list	l = [1,'h',"hola", 3]	Listas	SÍ
dict	d = "a":1 (KEY: VALUE)	Diccionarios	SÍ
set	s = 1, 2	Falta de descripción	SÍ
bytearray	b = bytearray(b"abc")	Falta descripción	SÍ

La **Mutabilidad** es una herramienta de seguridad y rendimiento y hasta concurrencia.

Objeto Mutable

Si se crean dos objetos mutables iguales, dos dict(), por ejemplo, cada variable referencia a su propio objeto.

Inmutabilidad

Si se crean dos objetos inmutables iguales, dos int() = 10, por ejemplo, ambos apuntan al mismo objeto. Eso permite evitar crear dos variables iguales, aumentando la seguridad. Si una variable cambia (pasa a valer 11), entonces Python crea un nuevo objeto con el valor actualizado y la variable reasigna la referencia al nuevo objeto.

En el caso de concurrencia, Estos objetos son super seguros por definición, ya que en sí son copias. Con ellos, no existen las **condiciones de competencia**.

Estructuras de control

```
if x > 0:
    code

for i in [something]:
    code
```

Al usar la sentencia de control "in", se puede iterar dentro de estructuras como rangos, listas, cadenas, diccionarios, etc.

Funciones

```
def suma(a, b):  
    return a + b
```

Si se pasa un objeto **immutable** a una función, este se pasa por valor, lo que quiere decir que se hace una copia de este objeto, por lo que los cambios en este objeto no ven reflejados en el objeto original. Por otro lado al pasar un objeto **mutable** a una función, este se pasa por referencia, por lo que este puede cambiar dentro de la función.

Listas y diccionarios

```
lst = [1,2,3]  
d = {"a": 1}
```

Comprensiones

```
[x*x for x in range(5)]
```

Clases

```
class A:  
    def __init__(self, x):  
        self.x = x
```

Entornos y paquetes

- `pip install pkg`
- `python -m venv venv`
- `source venv/bin/activate`